

Analisi del Report – Modulo 1 – SIMD

Walkthrough completo del report, sezione per sezione

Partitioned Hash Join with Duplicates

SPM – Università di Pisa – Prof. M. Torquati – A.A. 2025-2026

Walkthrough completo del report, sezione per sezione dall'inizio alla fine, con la motivazione di ogni scelta, la derivazione di ogni numero, la lettura di ogni grafico/tabella, il confronto con baseline/modulo precedente, i *risultati misurati* dove eseguibili senza cluster, e una sezione finale di ulteriori esperimenti. La struttura segue l'ordine del report originale, così puoi presentarlo scorrendo il PDF mentre mostri il documento al prof.

Contents

Come usare questo documento	2
1 Design Choices (sezione 1 del report)	2
1.1 Mapping Function	2
1.2 Data Layout and P	3
2 Auto-vectorization (sezione 2 del report)	3
3 AVX2 Intrinsics (sezione 3 del report)	3
4 Performance Analysis (sezione 4 del report)	3
4.1 Results	4
4.2 Why the Speedup is Limited (il cuore del report)	4
4.3 Autovec vs AVX2 Intrinsics	4
4.4 Parameter Sweeps	4
4.5 Measurement Stability	4
5 CUDA (sezione 5 del report)	5
6 Conclusions (sezione 6 del report)	5
7 Ulteriori esperimenti da fare	5

Come usare questo documento

Segue l'ordine del report (Design Choices → Auto-vectorization → AVX2 Intrinsics → Performance Analysis → CUDA → Conclusions). Per ogni sezione: cosa dice il report, perché, da dove vengono i numeri, come difenderlo. I box *Risultato misurato* contengono esperimenti che ho eseguito davvero (su una macchina sandbox, non sul cluster: i valori assoluti non sono quelli di node09, ma le *conclusioni qualitative* – es. bilanciamento della hash – sono valide ovunque).

1 Design Choices (sezione 1 del report)

Il report apre dichiarando il compito: dati N chiavi `uint64_t` e P partizioni (potenza di due), calcolare $\text{part_id}[i] = h(\text{keys}[i]) \in [0, P)$ per ogni chiave. È la sola *partition mapping kernel*, isolata dal join.

Motivazione della scelta

Perché isolare il kernel. Il join completo mescola fasi memory-bound (scatter) e compute-bound (probe in cache): misurando il solo map si ottiene un punto pulito sul roofline e si attribuisce ogni effetto SIMD senza confondersi con le altre fasi. È metodologia, da dichiarare.

1.1 Mapping Function

Il report sceglie una Fibonacci multiply-shift hash su 32 bit:

$$h(k) = ((k_{\text{lo}} \oplus k_{\text{hi}}) \cdot A_{32}) \gg (32 - \log_2 P), \quad A_{32} = 0x9E3779B9 = \lfloor 2^{32}/\varphi \rfloor.$$

Il report cita Knuth (hashing by multiplication) e Ferragina (multiply-add-shift universal hashing, qui con $b = 0$), e afferma che la costante aurea ha buone proprietà di dispersione dei bit.

Da dove viene il numero (derivazione)

A_{32} . $\varphi = (1 + \sqrt{5})/2 \approx 1.6180339887$; $2^{32}/\varphi \approx 2\,654\,435\,769.5 \Rightarrow \lfloor \cdot \rfloor = 0x9E3779B9$. È la stessa costante (a 64 bit) di `splitmix64`.

Motivazione della scelta

Tre ragioni, ognuna una possibile domanda: (1) *dispersione* – moltiplicare e prendere i bit *alti* fa dipendere l'output da tutti i bit dell'input; (2) *vettorizzabilità a 32 bit* – `vpmulld` fa 8 mul `uint32` in una istruzione, mentre una mul a 64 bit andrebbe emulata con ~ 3 `vpmuludq+shift+add`, rendendo l'AVX2 a 64 bit più lento dello scalare; (3) *modulo gratuito* – con P pot. di due, $h \bmod P$ è un right-shift. Lo XOR-fold mescola i 32 bit alti nei bassi prima della mul.

Il report afferma che su 10^8 chiavi con $P = 256$ la distribuzione misurata dà $\text{max}/\text{atteso} = 1.005$.

Risultato misurato (sandbox locale, non cluster)

Ho riprodotto la verifica del bilanciamento con la *stessa* hash e lo *stesso* generatore `splitmix64`, $N = 10^8$, $P = 256$:

chiavi	Fibonacci (max/atteso, Gini)	$k \bmod P$ (max/atteso, Gini)
uniformi	1.0035 , 0.0009	1.0050, 0.0009
strutturate (16 bit bassi a 0)	1.0046 , 0.0009	256.0 , 0.996

Su chiavi uniformi entrambe vanno bene (riproduco il ≈ 1.005 del report). Su chiavi *strutturate* Fibonacci resta bilanciata, mentre $k \bmod P$ collassa: tutte le 10^8 chiavi in una sola

partizione (Gini ≈ 1). **Questo è il motivo per cui si è scelta Fibonacci e non il reference $k \bmod P$** , ed è un risultato indipendente dall'hardware – vale identico sul cluster.

1.2 Data Layout and P

Il report mette chiavi e `part_id` in array separati contigui, stride unitario, allineati a 32 B (`posix_memalign`); P potenza di due.

Motivazione della scelta

SoA + stride unitario $\Rightarrow$ ogni cache line caricata contiene solo byte utili e gli accessi sono contigui: condizione per load/store vettoriali allineate (`vmovdqa`). In un kernel bandwidth-bound non sprecare byte di linea = non sprecare banda. L'allineamento a 32 B evita le penalità di split-line sulle load AVX2; P pot. di due rende il modulo uno shift.

2 Auto-vectorization (sezione 2 del report)

Il report compila lo *stesso* sorgente in due binari: baseline (`-fno-tree-vectorize`) e autovec (`-O3 -march=native -mavx2 -ftree-vectorize`), con `-fopt-info-vec-*` per la diagnostica. Riporta che GCC 12.2 vettorizza il loop emettendo due versioni – una a 32 B (`ymm`, $8 \times \text{uint32}$) e una a 16 B (`xmm`, fallback SSE) – e sceglie a runtime; vettorizza anche i loop di checksum e generazione. Il report `vec-missed` è vuoto.

Come difenderlo all'orale

“Perché GCC vettorizza?” Condizioni soddisfatte: trip count noto, nessuna dipendenza loop-carried, nessuna chiamata nel corpo, `__restrict__` (no aliasing), stride unitario, mul a 32 bit nativa. `vec-missed` vuoto $\Rightarrow$ nessun loop tentato e fallito: il codice è scritto apposta per essere vettorizzabile.

3 AVX2 Intrinsics (sezione 3 del report)

Il report descrive la kernel a mano che processa 8 chiavi/iterazione: due load `ymm` (4+4 chiavi), XOR-fold per lane, poi l'impacchettamento degli 8 valori a 32 bit con `permutevar8x32` (indici `[0,2,4,6,1,3,5,7]`) e `permute2x128` (selettore `0x20`), un `vpmulld`, un right-shift, una store a 256 bit; coda scalare per gli $N \bmod 8$. Verifica per checksum FNV-1a contro un riferimento scalare interno.

Come difenderlo all'orale

Perché le permute. Dopo lo XOR-fold il valore utile è nei 32 bit bassi di ogni lane da 64; AVX2 non ha un narrowing $4 \times 64 \rightarrow 4 \times 32$ economico, quindi `vpermd` porta i dword utili nella metà bassa e `vperm2i128` cuce le due metà in un `ymm` di 8 valori, pronti per `vpmulld`. La coda scalare usa la stessa `hash_key` $\Rightarrow$ output identico bit a bit, ed è ciò che il checksum FNV-1a (ordine-sensibile) verifica.

4 Performance Analysis (sezione 4 del report)

Setup: node09 (AMD EPYC 7301, Zen 1, DDR4-2666), allocazione SLURM esclusiva, 11 ripetizioni, mediana.

4.1 Results

N	Baseline (Mkeys/s)	Autovec (S)	AVX2 (S)
10^6	929	$1.48\times$	$1.37\times$
10^7	915	$1.39\times$	$1.28\times$
5×10^7	908	$1.45\times$	$1.29\times$
10^8	918	$1.43\times$	$1.31\times$
2×10^8	908	$1.46\times$	$1.27\times$

Table 1: $P = 256$. Throughput baseline ~ 915 Mkeys/s, autovec $\sim 1.43\times$, AVX2 $\sim 1.30\times$.

4.2 Why the Speedup is Limited (il cuore del report)

Il report calcola $I = 4/12 \approx 0.33$ ops/byte e colloca il kernel sulla rampa di banda del roofline; afferma che l'autovec raggiunge 15.7 GB/s, il 96% del tetto single-core misurato (~ 16.4 GB/s).

Da dove viene il numero (derivazione)

Da throughput a banda. 12 B/chiave (8 read + 4 write). Autovec: $1310 \frac{\text{Mkeys}}{\text{s}} \times 12 \text{ B} = 15.7 \text{ GB/s} \Rightarrow 15.7/16.4 \approx 96\%$. Baseline $\approx 11.0 \text{ GB/s}$, AVX2 $\approx 14.4 \text{ GB/s}$. **Intensità:** $I = 4/12 \approx 0.33$, sotto il ginocchio (5–10) $\Rightarrow$ memory-bound. Il SIMD accelera il compute, non la banda; lo speedup misura solo una migliore saturazione.

4.3 Autovec vs AVX2 Intrinsics

Il report osserva (controintuitivo) che l'autovec batte l'AVX2 a mano (15.7 vs 14.4 GB/s).

Come difenderlo all'orale

In regime memory-bound conta quanto bene riempi le richieste in volo (scheduling, unrolling, registri): GCC autovettorizzando ha libertà che le intrinsics rigide non danno. Prova: CoV 0.4% (autovec) vs 3.5% (AVX2). Le intrinsics convengono se compute-bound; qui no.

4.4 Parameter Sweeps

Il report mostra tre sweep: **N** (speedup piatto $\Rightarrow$ memory-bound a ogni taglia), **P** (throughput indipendente da $P \in [2, 1024]$, cambia solo lo shift), **key-space** (throughput invariante alla densità di duplicati).

Da dove viene il numero (derivazione)

Cos'è la "densità" (key_space). key_space = ampiezza dell'universo chiavi; piccolo $\Rightarrow$ molti duplicati. La kernel è branchless, costo costante per chiave $\Rightarrow$ né traffico né calcolo cambiano: throughput invariante. *Qui* la densità non conta (giusto); conterà nel *join* dove decide quante coppie produci.

4.5 Measurement Stability

Il report: tutti i 55 punti con CoV $< 5\%$; autovec il più stabile (CoV medio 0.4%), AVX2 più variabile (3.5%) – coerente con il fatto che lo scheduling di GCC produce accessi più deterministici. È la base statistica che legittima il confronto.

5 CUDA (sezione 5 del report)

Fase	Tempo (ms)	Banda (GB/s)	%
H→D	63.2	12.7	65%
Kernel	1.5	808	2%
D→H	32.3	12.4	33%
End-to-end	97.0	—	100%

Table 2: $N = 10^8$, A30, host pinned, 256 thread/blocco.

Da dove viene il numero (derivazione)

808 GB/s. $12B \times 10^8 = 1.2 \text{ GB} / 1.5 \text{ ms} = 800 \text{ GB/s} = 81\%$ del picco HBM2 (993). PCIe = 95.5 ms su 97 = 98%. End-to-end $10^8/97\text{ms} = 1031 \text{ Mkeys/s}$ vs 918 baseline $\Rightarrow 1.12\times$. La GPU non conviene: comp/comm ratio pessimo; rende solo con dati già on-device o pipeline successive su GPU.

6 Conclusions (sezione 6 del report)

Il report chiude: kernel da manuale memory-bound; autovec al 96% del tetto, niente margine single-thread; la scelta della hash a 32 bit (vpmulld) sopra una a 64 bit è stata critica – con 64 bit l’AVX2 sarebbe stato più lento dello scalare.

Come difenderlo all’orale

Riassunto in 4 frasi. (1) Memory-bound: $I = 0.33$, autovec al 96% del tetto (lo derivo dai Mkeys/s). (2) 32 bit per vpmulld; a 64 bit l’AVX2 sarebbe più lento dello scalare. (3) L’autovec batte le intrinsics perché in regime di banda conta lo scheduling del compilatore. (4) GPU inutile: PCIe = 98% del tempo.

7 Ulteriori esperimenti da fare

Esperimento di supporto

- [FATTO, vedi sopra] Bilanciamento Fibonacci vs $k \bmod P$**
Come: istogramma occupazione su chiavi uniformi e strutturate
Misura: max/atteso, Gini
Atteso: Fibonacci ≈ 1.00 sempre; $k \bmod P$ collassa su chiavi strutturate
Dimostra: il perché del cambio di mapping – già eseguito, risultati nel box teal.
- Counterfactual hash 64 bit**
Come: ricompila l’AVX2 con mul 64×64 emulata
Misura: Mkeys/s e GB/s vs scalare
Atteso: AVX2 64-bit $\leq$ scalare
Dimostra: perché si è scelto 32 bit (richiede il cluster per i tempi).
- Ancorare il tetto di banda**
Come: likwid-bench -t copy o STREAM single-core su node09
Misura: GB/s di picco
Atteso: $\approx 16.4 \text{ GB/s}$
Dimostra: giustifica il “96% del tetto”.
- Non-temporal stores**
Come: `_mm256_stream_si256` sull’output

Misura: GB/s

Atteso: guadagno $\lesssim 25\%$ sulle scritture

Dimostra: che anche l'ottimizzazione giusta e' limitata dal regime memory-bound.

5. **Multi-thread del map**

Come: OpenMP su 1–16 core di un socket

Misura: GB/s aggregati vs core

Atteso: plateau a banda satura

Dimostra: il tetto di banda dell'intero socket.

6. **Contatori HW**

Come: `perf stat -e cache-misses,LLC-load-misses`

Misura: miss rate, traffico LLC

Atteso: dominato dal DRAM

Dimostra: empiricamente il memory-bound.

Collegamento alla teoria

Roofline e intensità operativa; SIMD auto-vec vs intrinsics, allineamento, coda scalare (L6–L7); SIMT/coalescing e comp/comm ratio (L8); SoA vs AoS; STREAM come tetto.